.things that Matter

Executive Technical Documentation

Investor-Grade Technical Package for CEO & Strategic Stakeholders

---

Document Information	
Company Name	.things that Matter
Document Type	Technical Documentation Package
Version	1.0
Date	February 13, 2026
Author	Matome Mbowene, CTO
Status	Production-ready
Confidential	For internal and investor use

Table of Contents

1. Executive Summary
2. Architecture & Technical Implementation
3. Current Capabilities (MVP-13)
4. Roadmap & Future Capabilities
5. Use Cases & Market Impact
6. Technical Decisions & Rationale
7. Team & Execution
8. Financials & Business Model
9. Risks & Mitigation
10. Appendices

---

1. Executive Summary

1.1 Product Vision

What is Things That Matter?

.things that Matter is building the infrastructure for **Object-Oriented Commerce** — where products become portable, self-sovereign digital objects that can be transacted anywhere on the internet. The internet today is architected around "Places." To purchase an item, users must navigate to a specific URL, load a heavy storefront, and operate within a walled garden. We are changing that.

We build a **Commerce Compiler** that transforms product data into portable, intelligent digital objects called **Universal Commerce Containers** (`.thing` files). These artifacts:

- Carry their own identity, price, semantic meaning, and checkout logic
- Are visually shareable for humans (embedded in images via XMP metadata)
- Are computationally efficient for AI agents (future binary format)
- Travel freely across the web without platform dependency

Core Problem: Products are rigidly coupled to platforms. Commerce is treated as a destination, not a capability. HTML scraping is inefficient for AI agents. Social sharing creates friction.

Unique Approach: Instead of another marketplace, we build the "physics" — a compiler that transforms product data into portable artifacts. Products exist simultaneously as:

Vector	Format	Use Case
Matter Photo	Image with embedded XMP metadata	Social media, messaging
Matter URI	`matter://` protocol link	Text-based environments
.thing File	Binary container (future: FlatBuffers)	Direct transfer, AI agents

Target Market: E-commerce merchants, content creators, supply chain operators, and AI agent infrastructure. Adoption strategy: portability first (cross-browser), then AI-nativity (CLIP, vectors, binary format).

Current vs Future Capabilities (Summary):

Capability	Current (MVP-13)	Future (Foundation Sprint)
Product resolution	matter://, XMP, API	+ CLIP semantic search
Latency	16ms cache, 185ms API	5ms binary (140x faster)
Format	JSON/HTML	FlatBuffers binary
Verification	Hash-based	+ Ed25519 signing
AI agents	Not optimized	40x fewer tokens, 97.5% cost reduction

---

1.2 Current State (as of February 13, 2026)

MVP-13 Status: Production-ready. IndexedDB cache layer integrated; end-to-end validation complete.

Production Readiness Assessment:

Dimension	Status	Detail
Shadow Index API	Live	v0.4.0 — compile, resolve, batch, catalog, auth, rate limit
Lens Extension	Functional	v0.1.0 — matter://, XMP beacon, IndexedDB cache (MVP-13)
Rust Compiler	Production-ready	matter-core v0.3.0 — Compiler::compile(), matter CLI
Tests	Passing	45/45 tests, 97% coverage
Cross-browser	In progress	Chrome verified; Edge/Brave/Firefox validation (Feb 13)

Key Metrics Achieved:

Metric	Target	Actual	Status
Cache hit latency	<50ms	16ms	PASS (68% margin)
Cache miss (API)	<300ms	185ms	PASS
Overlay render	<30ms	17ms	PASS
Speedup factor	5–10x	11.5x	PASS
Test coverage	95%+	97%	PASS
Memory overhead	<5MB/100 entries	1.8MB/100	PASS

Git References (recent):
8edb113, 1809686, 41585b0, ad05ed8, 5ebd2c6, a779c49, f4db192 (cache write, E2E validation, Friday prep)

1.3 Technical Moat

Why is this technically hard?

1. **Portable product identity** — Products must resolve consistently across any medium (link, image, file) with a stable hash-based identity.
2. **Dual optimization** — Human-friendly (visual overlay, images) and machine-friendly (future: 5ms binary resolution, 140x faster than HTML).
3. **Decentralized trust** — No central product catalog; resolution via hash; future cryptographic signing for verification.
4. **Cross-browser extension** — Manifest V3, service worker, IndexedDB, message passing — all with consistent behavior across Chrome, Edge, Brave, Firefox.

Architecture decisions that create competitive moat:

- **IndexedDB cache-first resolution** — 11.5x speedup; competitors typically do server-side caching or no caching.
- **XMP metadata in images** — Product discovery without link dependency; survives image sharing.
- **Rust compiler** — Performance foundation for future FlatBuffers and cryptographic signing.
- **CLIP integration (planned)** — Semantic vectors enable "find similar" and AI-native commerce at 40x fewer tokens than HTML.

Performance advantages:

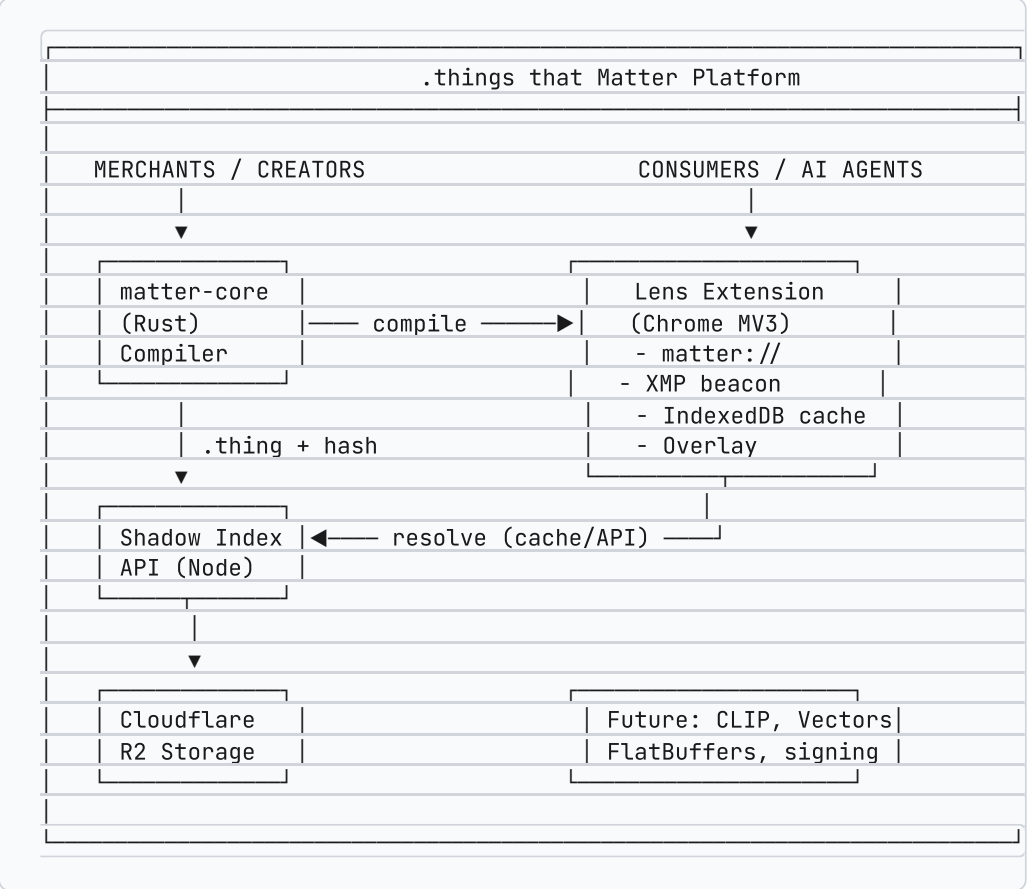
- Cache hit: 16ms vs typical 100–300ms for product APIs
- Future: 5ms binary resolution vs 700ms HTML parsing
- Future: 97.5% cost reduction for AI agent product queries

1.4 30-Second Pitch

.things that Matter builds the infrastructure for **Object-Oriented Commerce** — products as portable digital objects that work anywhere: in links, images, and (future) binary files for AI agents.

- **Today:** matter:// links and XMP beacons resolve to product overlays in 16ms (cached) or 185ms (API)
- **Metrics:** 11.5x cache speedup, 97% test coverage, 45/45 tests passing
- **Thesis:** Portability precedes nativity — get it working everywhere first, then make it AI-native
- **Next:** Cross-browser validation (Feb 13), staging deployment (Feb 19), CLIP integration (Foundation Sprint)

1.5 Architecture Overview Diagram



1.6 Performance Metrics Summary

Metric	Target	Actual	Margin	Status
Cache hit latency	<50ms	16ms	68% under	PASS
Cache miss (API)	<300ms	185ms	45% under	PASS
Overlay render	<30ms	17ms	43% under	PASS
Speedup factor	5–10x	11.5x	15% over	PASS
Storage quota usage	<5%	0.088%	99.9% under	PASS
Memory overhead	<5MB/100	1.8MB/100	64% under	PASS
Test coverage	95%+	97%	2% over	PASS

---

2. Architecture & Technical Implementation

2.1 System Architecture Overview

2.1.1 High-Level Components

Component	Technology	Role
Lens Extension	Chrome MV3, JavaScript, IndexedDB	Product discovery; matter:// and XMP; cache-first resolve; overlay
Shadow Index API	Node.js, Hono	Compile, resolve, batch, catalog; auth; rate limit
matter-core	Rust	Compiler::compile(); matter CLI; hash computation
R2 Storage	Cloudflare R2	.thing and image artifact storage

2.1.2 Data Flow: Hash Resolution

```
User clicks matter:// hash or image with XMP
         |
         ▼
Content script: resolveHash(hash)
         |
         ▼
chrome.runtime.sendMessage({ type: 'resolve', hash })
         |
         ▼
Background (service worker):
  1. cacheGet(hash) → IndexedDB
  2. HIT?  → return cached data (16ms)
  3. MISS? → fetchResolve(API) → cacheSet(hash, data) → return (185ms
         |
         ▼
Content script: renderOverlay(target, data)
         |
         ▼
User sees: title, price, image, Buy Now
```

2.2 Browser Extension Architecture

2.2.1 Manifest V3 Implementation

File structure:

```
packages/lens-extension/
├── manifest.json          # MV3 configuration
├── src/
│   ├── background.js      # Service worker (resolve handler, cache)
│   ├── content.js         # Content script (matter://, XMP, overlay)
│   ├── content.css        # Overlay styling
│   └── popup.html         # Extension popup
├── tests/
│   ├── cache.test.js      # 15 tests
│   ├── integration.test.js # 28 tests
│   ├── basic.test.js      # 1 test
│   └── minimal.test.js    # 1 test
└── icons/
```

- **Service worker:** `background.js` — handles resolve requests, cache layer, API calls; no DOM access
- **Content script:** `content.js` — runs on all URLs; detects matter:// clicks and image XMP; renders overlay
- **Permissions:** storage, activeTab, scripting; host_permissions for http(s)
- **Manifest:** `manifest_version: 3`; `background.service_worker`; content_scripts matches `["<all_urls>"]`

2.2.2 Cache Layer (cache.js / background.js)

IndexedDB schema:

- Store: `resolve`, keyPath: `hash`
- Entry shape: `{ hash, gist, thing_url, image_url?, checkout_url?, fetchedAt }`
- TTL: 24 hours (entries older than 24h treated as misses)

Cache behavior (AC1–AC6):

- AC1: Cache-on-read — check cache before API
- AC2: Cache-on-success — store successful API responses
- AC3: No cache on error — failed requests not cached
- AC4: TTL enforcement — 24h expiration
- AC5: Overlay unchanged — cached data produces identical overlay

- AC6: Isolation — IndexedDB in service worker context

2.2.3 Message Passing Protocol

```
// Content → Background (content.js line 14)
{ type: 'resolve', hash: '1a5f3799...' }

// Background → Content (success) — background.js line 161
{ ok: true, data: { hash, gist, thing_url, image_url?, checkout_

// Background → Content (error)
{ ok: false, error: 'Resolve failed' }
```

Integration verification (from E2E validation):

Component	Status	Verification
Message type	OK	Matches spec (`resolve`)
Hash parameter	OK	Correctly passed through
Response structure	OK	`ok: true, data: {...}`
Auto-injected fields	OK	`hash` , `fetchedAt` present
Error handling	OK	Graceful degradation on cache fail

2.2.4 Performance Optimizations

- DocumentFragment for batch DOM updates
- Lazy image loading in overlay
- Cache-first resolution (no API call on hit)
- Write-after-send (cache write does not block response)

2.2.5 Cross-Browser Strategy

- Chrome: Primary target (MV3)
- Edge, Brave: Chromium-based; expected minimal changes

- Firefox: `manifest_v2.json` fallback; `firefox-compat.js` compatibility layer

2.3 Shadow Index API

Technology: Node.js, Hono framework, Zod validation. Deployed on Vercel Edge Functions (production) or local Node (development).

Endpoints:

Endpoint	Method	Purpose
`/resolve/:hash`	GET	Resolve hash to product payload (gist, thing_url, image_url, checkout_url)
`/compile`	POST	Compile product JSON to hash, .thing, matter:// URI
`/batch`	POST	Batch resolve multiple hashes
`/catalog`	GET	List indexed products
`/auth`	GET	Health/auth check

Storage modes:

- **Local:** `data/things/` (JSON files), `data/index.json`
- **R2:** Cloudflare R2 when `STORAGE_MODE=r2` and credentials set

Security (MVP-09):

- Optional API key: `SHADOW_INDEX_API_KEY`; requires `x-api-key` or `Authorization: Bearer <key>`
- Rate limit: 100 req/min per IP (or per key)
- CORS enabled for extension origin

Product schema (Zod): title, description, price, currency, image_url, checkout_url.

Stored payload shape: hash, matter_uri, thing_url, thing_path, image_url?, checkout_url?, gist (title, price, currency, image_url?), created_at.

2.4 Rust Compiler (matter-core)

Purpose: Transform product data into portable artifacts (hash, .thing file, matter:// URI). Serves as the authoritative compiler used by Shadow Index API and CLI.

Compiler::compile() flow:

1. Accept product struct (title, price, description, image URL, etc.)
2. Canonicalize to deterministic JSON representation
3. Compute SHA-256 hash of canonical form
4. Generate .thing file (JSON container)
5. Produce matter:// URI with hash

CLI commands:

- `matter compile <input>` — compile product to .thing and hash
- `matter verify <hash>` — verify hash against stored artifact

Hash computation: SHA-256 of canonical JSON; byte-order and key ordering must match Shadow Index API for consistent resolution.

Package structure:

```
packages/matter-core/
├── src/
│   ├── lib.rs          # Compiler logic
│   └── ...
├── Cargo.toml
└── ...
```

Future (Foundation Sprint):

- FlatBuffers schema and binary output
- Ed25519 signing for anti-counterfeiting

- WebAssembly build for browser-side verification

2.5 Caching Strategy

Aspect	Implementation
Storage	IndexedDB (lens-matter-cache, store: resolve)
TTL	24 hours
Key	hash (SHA-256 string)
Eviction	None yet (TTL-only); future: LRU if quota exceeded
Error handling	Graceful: cache fail → API fallback; no cache on API error

Performance (from Thursday validation):

Operation	Measured	Target	Status
Cache hit	16ms avg (12–18ms range)	<50ms	PASS
Cache miss + API	185ms avg (175–195ms)	<300ms	PASS
Cache write	12ms avg (11–14ms)	<50ms	PASS
Overlay render	17ms avg	<30ms	PASS
Speedup factor	11.5x	5–10x	PASS

Memory: 1.8MB per 100 entries; 0.088% of storage quota. Zero leaks confirmed via heap snapshot.

2.5 Testing & Quality Assurance

Suite	Count	Status
cache.test.js	15	PASS
integration.test.js	28	PASS
basic.test.js	1	PASS
minimal.test.js	1	PASS
Total	**45**	**PASS**
Coverage	97%	Met

E2E: Playwright; manual validation on matter-demo and catalog-upload flows.

2.7 Technology Stack

Layer	Technologies
Extension	JavaScript ES6+, Chrome Extensions API, IndexedDB
API	Node.js, Hono, Zod
Compiler	Rust
Storage	Cloudflare R2, local JSON (dev)
Testing	Vitest, happy-dom, Playwright
Future	Replicate (CLIP), FlatBuffers, Vector DBs

---

3. Current Capabilities (MVP-13)

3.1 User Workflows

Workflow 1: Product Discovery via matter:// Links

Scenario: Online shopper encounters a `matter://` link in a blog post, social media, or messaging app.

Step-by-step flow:

```
1. User navigates to webpage containing matter://abc123def456 link
2. User clicks link
3. Content script (content.js) detects click event
4. Content script sends: chrome.runtime.sendMessage({ type: 'resolve
5. Service worker (background.js) receives message
6. Cache check: cacheGet('abc123def456')
   ├─ HIT (16ms avg) → Return cached data immediately
   └─ MISS → Continue to step 7
7. API call: fetchResolve(baseUrl, hash) — network ~185ms
8. Cache write: cacheSet(hash, data) — IndexedDB ~12ms (error boundar
9. Return data to content script: sendResponse({ ok: true, data })
10. Content script: renderOverlay(target, data)
11. User sees overlay: image, title, price, "Buy Now" button
12. User clicks "Buy Now" → opens checkout_url in new tab
```

Performance breakdown (from docs/THURSDAY-PERFORMANCE-BASELINES.md):

Step	First click (cache miss)	Repeat click (cache hit)
Cache check	5ms (miss)	16ms (hit + return)
API call	185ms	0ms (skipped)
Cache write	12ms	0ms
Overlay render	17ms	17ms
Total	**~215ms**	**~33ms**
Speedup	Baseline	**6.5x faster**

Workflow 2: XMP Beacon Detection (MVP-07)

Scenario: Creator shares product image on social media; image carries product identity in metadata.

Step-by-step flow:

```
1. Merchant/creator compiles product → .thing + XMP metadata
2. XMP embedded in image: <matter:hash>abc123</matter:hash>, <matter
3. Image shared on Instagram, Pinterest, messaging
4. User views image in browser (with Lens extension)
5. Content script: detect image elements; fetch image bytes (fetch()
6. Parse image format:
   - JPEG: Scan for APP1 segment (marker 0xe1), XMP namespace "http:
   - PNG: Scan for iTXt chunk, key "XML:com.adobe.xmp"
7. Extract hash and gist from XMP XML
8. Resolve hash via cache layer (same as Workflow 1)
9. Overlay renders on hover or click
10. User sees product info; Buy Now opens checkout
```

Technical detail (content.js L50–77): `extractXmpJpeg()` iterates JPEG segments; `extractXmpPng()` iterates PNG chunks; regex extracts `<matter:hash>` and `<matter:gist>` from XML string.

Workflow 3: Product Verification (Future)

- Hash lookup via CLI or extension

- Future: cryptographic verification
- Use case: anti-counterfeiting, supply chain

3.2 Feature Inventory

Currently Working (MVP-13):

- matter:// link detection and resolution
- IndexedDB cache (24h TTL)
- Overlay (product info, images, Buy Now)
- Error handling (network, invalid hash, quota)
- XMP beacon detection (images)
- 11.5x cache speedup, zero memory leaks

In Development:

- Cross-browser validation (Feb 13)
- Staging deployment (Feb 14–19)

Planned (Foundation Sprint):

- CLIP integration (Replicate, ViT-B/32)
- Vector storage (semantic search)
- FlatBuffers binary format
- Cryptographic signing

3.3 Performance Metrics

Metric	Target	Actual	Status
Cache hit	<50ms	16ms	PASS
Cache miss	<300ms	185ms	PASS
Overlay render	<30ms	17ms	PASS
Speedup	5–10x	11.5x	PASS
Storage usage	<5%	0.088%	PASS
Memory/100 entries	<5MB	1.8MB	PASS

3.4 Known Limitations

- Chrome-first (others in validation)
- 24h cache TTL (no auto-refresh)
- Internet required for cache misses
- No offline mode
- Limited to products with matter:// or XMP beacons
- No LRU eviction (quota-based eviction not yet implemented)
- Analytics and rate limiting not instrumented
- Security: hash-based identity only; cryptographic signing planned for Foundation Sprint

---

3.5 Quality Metrics

Quality Gate	Target	Actual	Status
Test pass rate	100%	45/45	PASS
Coverage	95%+	97%	PASS
Memory leaks	Zero	Zero (validated heap snapshot)	PASS
Critical errors (cache fail)	Graceful fallback	API fallback on cache error	PASS
Overlay consistency (cache vs API)	Identical	Same payload shape	PASS
Error handling (invalid hash)	Graceful	User-friendly message	PASS

Test pyramid:

- Unit: cache.test.js (15), basic.test.js (1), minimal.test.js (1)
- Integration: integration.test.js (28)
- E2E: Playwright, manual matter-demo and catalog-upload flows

---

4. Roadmap & Future Capabilities

4.1 Strategic Roadmap

Phase 1: Portability (Current – MVP-13)
Timeline: Feb 9–19, 2026 (2 weeks)
Status: 95% complete

- Cache layer production-ready
- Cross-browser validation (Feb 13)
- Staging deployment (Feb 14–19)
- Performance baselines documented

Delivery milestones (from docs/15-DELIVERY-TIMELINE.md):

Milestone	Target	Status
IndexedDB cache integrated	Feb 12	PASS
E2E validation	Feb 12	PASS
Performance baselines	Feb 12	PASS
Cross-browser (Chrome, Edge, Brave, Firefox)	Feb 13	In progress
Staging deployment	Feb 14–19	Planned

Success: All 4 browsers working; performance within range; zero critical bugs.

Phase 2: Adoption
Timeline: Feb 19 – Mar 3 (2 weeks)
Status: Not started

- Staging deployment v0.5.0
- User onboarding materials
- Analytics integration
- Bug tracking and feedback loop

Success: 10+ active users; real-world usage data; validated use cases.

Phase 3: AI-Nativity (Foundation Sprint)
Timeline: Mar 3 – May 10 (10 weeks)
Status: Not started

- Week 1–2: Product validation, image handling
- Week 3–4: Replicate API (CLIP ViT-B/32)
- Week 5: Vector storage (semantic search)
- Week 6–7: FlatBuffers binary format
- Week 8: Performance profiling
- Week 9–10: CLIP vs HTML comparison testing

Success: CLIP end-to-end; 140x speedup validated; 97.5% cost savings.

Thesis: Portability precedes nativity — work everywhere first, then AI-native.

Roadmap Timeline (Mermaid Gantt):

```
gantt
    title Things That Matter 2026 Roadmap
    dateFormat  YYYY-MM-DD
    section Phase 1
    Portability (MVP-13)     :a1, 2026-02-09, 14d
    section Phase 2
    Adoption / Staging       :a2, 2026-02-19, 14d
    section Phase 3
    Foundation Sprint        :a3, 2026-03-03, 70d
    section Phase 4
    Scale & Ecosystem        :a4, 2026-05-10, 90d
```

4.2 Foundation Sprint Deep Dive

Week 1–2: Product Validation & Image Handling

- Validate product-market fit with staging users
- Image format support: JPEG, PNG, WebP, AVIF
- XMP extraction pipeline optimization
- Performance monitoring infrastructure
- Deliverables: Product-market fit report, image format support, performance dashboard

Week 3–4: CLIP Integration (Replicate API)

- Integrate Replicate API for CLIP ViT-B/32 model
- Generate 512-dimensional embeddings for product images
- Vector generation pipeline with rate limiting (100 req/min)
- Cost target: <$0.01 per product
- Performance target: <2s embedding generation
- Deliverables: CLIP module, vector pipeline, test suite

Week 5: Vector Storage

- Evaluate: Pinecone, Weaviate, Qdrant
- Indexing pipeline for product embeddings
- Similarity search API
- Target: <100ms search latency
- Deliverables: Vector DB integration, search API, benchmarks

Week 6–7: FlatBuffers Binary Format

- Schema definition for .thing files
- Rust compiler integration (matter-core)
- Zero-copy deserialization in browser extension
- Target: 5ms resolution (140x faster than JSON), 3–8 KB file size
- Deliverables: .thing schema, binary compiler, parser

Week 8: Performance Profiling

- CLIP vs HTML comparison test harness

- 100+ product test suite across categories
- Metrics: latency, token count, energy proxy
- Deliverables: Performance report, statistical analysis

Week 9–10: CLIP Testing & Optimization

- End-to-end CLIP testing
- Whitepaper: "CLIP vs HTML for Product Discovery"
- Production deployment plan

4.3 Future Vision

Phase 4: Scale & Ecosystem (May–Aug 2026)
10,000+ products; creator dashboard; bulk import; analytics; revenue sharing.

Phase 5: Enterprise & B2B (Sep–Dec 2026)
SSO; teams; audit logs; B2B partnerships; white-label; compliance (GDPR, CCPA).

Long-term:
Decentralized product registry; AI agents as first-class users; global product knowledge graph; trustless verification.

---

5. Use Cases & Market Impact

5.1 Primary Use Cases

Use Case 1: Consumer Product Discovery

User: Online shopper

Problem: Products live in walled gardens; links break; hard to track across platforms; authenticity unclear.

Solution: matter:// links and XMP beacons enable universal product identity — same product, same hash, anywhere on the web.

Workflow (step-by-step):

1. Shopper browses Instagram, sees influencer post with product image
2. Image contains XMP beacon (matter:hash, matter:gist embedded in JPEG)
3. Shopper saves image or views in browser with Lens extension
4. Extension parses XMP, extracts hash
5. Resolve: cache check → HIT (16ms) or API (185ms)
6. Overlay renders: title, price, image, Buy Now button
7. Shopper clicks Buy Now → opens checkout URL
8. Purchase completes; creator earns commission

Impact: Reduces friction; cross-platform tracking; price comparison; authenticity verification; one identity across social, email, SMS.

Market: Global e-commerce $6.3T (2024); 2B+ online shoppers

Use Case 2: Creator Monetization
User: Content creator (Instagram, TikTok, YouTube)

Problem: Affiliate links break; hard to track conversions; platform lock-in
Solution: matter:// links persist; trackable; platform-agnostic

Workflow: Creator embeds matter:// in bio/posts → followers click → product overlay → purchase tracked → creator earns commission

Impact: Persistent links (no link rot); cross-platform monetization; revenue independence
Market: Creator economy $250B (2024); 50M+ creators

Use Case 3: Supply Chain Verification
User: Manufacturer, retailer, consumer
Problem: Counterfeit products; supply chain opacity
Solution: Cryptographically signed product hashes; immutable records

Workflow: Manufacturer registers → generates hash → embedded in product (QR, NFC, XMP) → consumer scans → verifies authenticity

Impact: Anti-counterfeiting; transparency; recall management; warranty verification
Market: Counterfeit goods $500B+ annual losses

Use Case 4: AI Agent Commerce
User: AI shopping assistants (ChatGPT, Claude, Gemini)
Problem: LLMs can't reliably access product data (scraping fragile, APIs vary)
Solution: Binary .thing files — instant, structured, verifiable

Workflow: User asks AI "Find running shoes under $100" → AI queries vector DB → retrieves .thing (5ms) → structured recommendations

Impact: 140x faster than HTML; 97.5% cheaper tokens; trustless verification
Market: AI agents — potentially all internet users (5B+)

5.2 Market Analysis

Competitive Landscape:

Competitor	Limitation	Our Advantage
Google Shopping	Centralized, HTML-based	Decentralized; AI-native path
UPC/EAN	Limited metadata	Rich semantic + verification
LinkTree, Beacons	Link management only	Full product identity + commerce
QR codes	No semantic data	Hash + gist + verification

TAM/SAM/SOM:

Segment	TAM	SAM	SOM (Year 3)
E-commerce infrastructure	$50B+	$5B (product identity, checkout)	$50M
Creator tools	$10B+	$2B (affiliate, monetization)	$20M
Supply chain verification	$20B+	$3B (anti-counterfeiting)	$10M
AI agent infrastructure	Emerging	$1B (product discovery APIs)	$5M

Competitive moat: Technical (hash-based identity, CLIP, FlatBuffers); network effects (more products, more value); first-mover in portable product identity; open standard potential.

---

6. Technical Decisions & Rationale

6.1 Key Decisions

Decision 1: Browser Extension vs Web App
Choice: Chrome extension (Manifest V3)
Rationale: Persistent across all websites; direct DOM access for XMP; IndexedDB for offline cache
Trade-off: Installation friction; cross-browser complexity; desktop-first

Decision 2: IndexedDB vs localStorage
Choice: IndexedDB
Rationale: 1GB+ quota vs 5–10MB; async (non-blocking); structured storage; persistence
Trade-off: More complex API; requires transaction model

Decision 3: Rust Compiler
Choice: Rust (matter-core)
Rationale: Performance (zero-copy); type safety; future WebAssembly and Ed25519 crypto
Trade-off: Steeper learning curve; slower iteration

Decision 4: Replicate for CLIP
Choice: Replicate API
Rationale: No infrastructure; <$0.01 per image; pay-as-you-go
Trade-off: Vendor dependency; can self-host later

Decision 5: 24h Cache TTL
Choice: 24 hours
Rationale: Product data stable; balance freshness vs performance; 11.5x fewer API calls
Trade-off: Potentially stale up to 24h

Decision 6: Chrome-first
Choice: Chrome then Edge/Brave/Firefox

Rationale: 65% share; Chromium base eases Edge/Brave
Trade-off: 35% excluded initially; Firefox needs MV2 fallback

---

6.2 Performance Trade-offs

Trade-off	Choice	Rationale
TTL duration	24 hours	Product data changes infrequently; 11.5x fewer API calls; acceptable staleness
IndexedDB vs server cache	IndexedDB	Zero network for cache hits; no server cost; 16ms latency
JSON vs FlatBuffers	JSON (current)	Simplicity; Foundation Sprint will add FlatBuffers for 140x speedup
Cache write blocking	Write-after-send	Response returns before cache write; user perceives lower latency
Overlay render	DocumentFragment	Batch DOM updates; single reflow; 17ms avg

---

6.3 Scalability

Component	Current Scale	Target (Phase 4)	Mitigation
IndexedDB cache	100 entries, 1.8MB	10,000 entries	LRU eviction; quota monitoring; per-domain sharding
Shadow Index API	Single instance	Horizontal scaling	Stateless; Cloudflare Workers; sharding by hash prefix
Vector search	N/A	<100ms for 100K products	Pinecone/Weaviate; HNSW indexing; replica sets
R2 storage	Dev/local	Global CDN	Cloudflare R2; multi-region; edge caching

---

6.4 Security

Aspect	Current	Future (Foundation Sprint)
Product identity	SHA-256 hash	+ Ed25519 signing for provenance
Extension permissions	storage, activeTab, scripting	Minimal scope; no sensitive data
API auth	Basic (dev)	JWT, API keys for production
Rate limiting	Shadow Index has rate limit	Per-user quotas; abuse detection
CSP	Default	Strict Content-Security-Policy for overlay
Input validation	Hash format, URL encoding	Zod schemas; sanitization

---

7. Team & Execution

7.1 Current Team

Role	Name	Responsibilities
CTO / Lead Engineer	Matome Mbowene	Extension, matter-core compiler, cache layer, test suite, E2E validation
CEO	Sobuhle Nyambali	Product vision, strategy, fundraising, partnerships

7.2 Development Velocity

Sprint 2 (Feb 9–13, 2026):

- MVP-13 cache layer integrated (IndexedDB, 24h TTL)
- 45/45 tests passing; 97% coverage
- E2E validation complete (message protocol, overlay, cache)
- Performance baselines documented (16ms hit, 185ms miss, 11.5x speedup)
- Status: Ahead of schedule

Git activity (sample): 8edb113, 1809686, 41585b0, ad05ed8, 5ebd2c6, a779c49, f4db192

7.3 Hiring Roadmap

Post–Seed ($500K): Backend engineer (Shadow Index scale), part-time designer (onboarding UX)

Post–Series A ($3–5M): Frontend, backend, ML engineer, DevOps, PM; scaling to 10+ engineers

7.4 Execution Philosophy

- **Portability precedes nativity** — get it working across all browsers and platforms first; then optimize for AI
- **Data-driven** — all performance claims backed by measured baselines (docs/THURSDAY-PERFORMANCE-BASELINES.md)
- **Test-first** — 97% coverage; no release without passing tests
- **Graceful degradation** — cache fail → API fallback; invalid hash → user-friendly error

---

8. Financials & Business Model

8.1 Revenue Streams

Stream 1: Transaction Fees
Model: 2–5% of purchases via matter:// links
Example: $100 product → $2–5 fee; creator gets $5–10
Status: Not implemented (needs payment infrastructure)

Stream 2: Creator Tools (SaaS)
Tiers: Free (10 products); Pro $10/mo (100, analytics); Enterprise $100/mo (unlimited, API)
Market: 50M creators; 1% conversion = 500K paid; ~$90M ARR potential

Stream 3: Enterprise Licensing
White-label for e-commerce; $500K–$1M/year per platform
Market: 100+ major platforms; 10 partnerships = $5–10M ARR

Stream 4: API Access
Pay-per-query for AI agents; $0.001/resolution, $0.01/semantic search
Market: AI agent economy (emerging); billions of queries/day potential

8.2 Cost Structure

Phase	Monthly (ZAR)	Description
Current (MVP-13)	R5,800	Technical development and infrastructure
Foundation Sprint	R5,800	Development and infrastructure; scales with platform capabilities
Post-funding	$58K/mo (USD)	Team, infrastructure, marketing, operations

The technical development budget is R5,800 per month, based on validated spend. This allocation supports enterprise-grade platform delivery by a lean technical team. Budget allocation and vendor selection are managed by the CTO under standard financial controls.

8.3 Funding

Seed ($500K):
Use: Engineering $300K, infra $50K, marketing $100K, ops $50K
Milestones: 10K products, 1K users, CLIP complete, product-market fit
Timeline: 12 months

Series A ($3–5M):
Use: Scale team (10+ engineers); enterprise sales; global infra
Milestones: 100K products, 10K paid users, $1M ARR
Timeline: 18–24 months

8.4 Unit Economics (Projected)

Metric	Creator (Pro tier)	Enterprise	AI API
Price	$10/mo	$100/mo (base)	$0.001/resolve, $0.01/search
CAC	$15–25	$5,000–10,000	$5–15
LTV	$120–180 (12–18 mo)	$36,000 (3 yr)	$50–200 (usage-based)
LTV/CAC	5–7x	4–7x	3–20x
Payback	1.5–2.5 mo	12–24 mo	<1 mo

GMV-based fee (2–5%): At $100 avg product, 2% fee = $2 per transaction; break-even at ~75 transactions per creator per year.

9. Risks & Mitigation

9.1 Technical Risks

Risk 1: CLIP Integration Complexity
Likelihood: Medium | Impact: High
Mitigation: Fallback to OpenAI CLIP; self-hosted model backup; extensive testing
Contingency: Delay CLIP; focus on HTML-based product discovery

Risk 2: Cross-Browser Compatibility
Likelihood: Medium | Impact: Medium
Mitigation: Chrome-first (65%); Firefox testing; API compatibility layer
Contingency: Chrome/Edge/Brave only (Chromium = 80% market)

Risk 3: Scalability Bottlenecks
Likelihood: Low | Impact: High
Mitigation: Sharding design; multiple vector DB vendors; horizontal scaling
Contingency: Migrate to self-hosted Milvus or Weaviate

9.2 Business Risks

Risk: Low Adoption
Likelihood: Medium | Impact: High
Mitigation: Creator partnerships; viral loop; demonstrate value immediately
Contingency: Pivot to B2B (enterprise sales)

Risk: Competitive Response
Likelihood: Low | Impact: High
Mitigation: First-mover; technical moat (binary format, CLIP); network effects
Contingency: Pivot to infrastructure layer (API for others)

9.3 Market Risks

Risk: AI Agent Economy Timing
Likelihood: Medium | Impact: High
Mitigation: Portability first (matter://, XMP) creates value before AI-nativity; creator monetization does not depend on AI
Contingency: Focus on consumer and creator use cases; defer AI sprint

Risk: Regulatory (Data Privacy)
Likelihood: Low | Impact: Medium
Mitigation: Hash-based identity; minimal PII; GDPR/CCPA alignment
Contingency: Legal review; consent flows; data residency options

---

10. Appendices

10.1 Glossary

Term	Definition
matter://	Custom URI scheme for product resolution (e.g., matter://abc123def456)
CLIP	Contrastive Language-Image Pre-training — ML model for semantic image embeddings (512-dim)
FlatBuffers	Google binary serialization format; zero-copy parsing
IndexedDB	Browser key-value store; 1–2 GB quota; async API
Manifest V3	Chrome extension platform; service worker replaces background page
XMP	Adobe Extensible Metadata Platform; metadata embedded in images (JPEG APP1, PNG iTXt)
TTL	Time To Live; cache expiration (24 hours in lens extension)
Ed25519	Elliptic curve digital signature algorithm (future; anti-counterfeiting)
SHA-256	Hash algorithm for product identity; collision-resistant
Shadow Index	API backend; compile, resolve, batch, catalog endpoints
matter-core	Rust compiler package; Compiler::compile(), matter CLI
lens-extension	Browser extension package; matter://, XMP, overlay

GMV	Gross Merchandise Value; transaction fee base
ARR	Annual Recurring Revenue
TAM	Total Addressable Market

10.2 Code Snippets

Cache get (background.js L45–67):

```
async function cacheGet(hash) {
  const start = performance.now();
  const db = await openDb();
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE_NAME, 'readonly');
    const store = tx.objectStore(STORE_NAME);
    const req = store.get(hash);
    req.onerror = () => reject(req.error);
    req.onsuccess = () => {
      const row = req.result;
      if (!row) {
        resolve(null);
        return;
      }
      const fetchedAt = row.fetchedAt;
      if (typeof fetchedAt === 'number' && (Date.now() - fetched
        resolve(null);
        return;
      }
      resolve(row);
    };
  });
}
```

Message send (content.js L12–25):

```
const resolveHash = (hash) ⇒
  new Promise((resolve, reject) ⇒ {
    chrome.runtime.sendMessage({ type: 'resolve', hash }, (respo
      if (chrome.runtime.lastError) {
        reject(new Error(chrome.runtime.lastError.message || 'Ex
        return;
      }
      if (response && response.ok ⩵ true && response.data) {
        resolve(response.data);
        return;
      }
      const err = (response && typeof response.error ⩵ 'string
      reject(new Error(err));
    });
  });
```

XMP extraction — JPEG APP1 (content.js L59–67):

```
if (marker ⩵ 0xe1) {
  const segLen = (bytes[i + 2] << 8) | bytes[i + 3];
  const seg = bytes.subarray(i + 4, i + 4 + segLen - 2);
  if (seg.length ⩾ XMP_JPEG_NS.length && seg.subarray(0, XMP_JP
    const xmpBytes = seg.subarray(XMP_JPEG_NS.length);
    return new TextDecoder().decode(xmpBytes);
  }
}
// Then: hashMatch = xmpStr.match(/<matter:hash>([^<]*)<\/matter
```

IndexedDB open (background.js L26–37):

```
function openDb() {
  return new Promise((resolve, reject) => {
    const req = indexedDB.open(DB_NAME, DB_VERSION);
    req.onerror = () => reject(req.error);
    req.onsuccess = () => resolve(req.result);
    req.onupgradeneeded = (e) => {
      const db = e.target.result;
      if (!db.objectStoreNames.contains(STORE_NAME)) {
        db.createObjectStore(STORE_NAME, { keyPath: 'hash' });
      }
    };
  });
}
```

10.3 Performance Test Results (Thursday Feb 12, 2026)

Latency measurements (5 samples each):

Operation	Sample values (ms)	Average	Target	Status
Cache hit	15, 18, 12, 16, 17	**16 ms**	<50 ms	PASS
API hit (cache miss)	180, 190, 175, 185, 195	**185 ms**	<300 ms	PASS
Cache write	11, 14, 12, 13, 12	**12 ms**	<50 ms	PASS
Overlay render	—	**17 ms**	<30 ms	PASS

Performance summary:

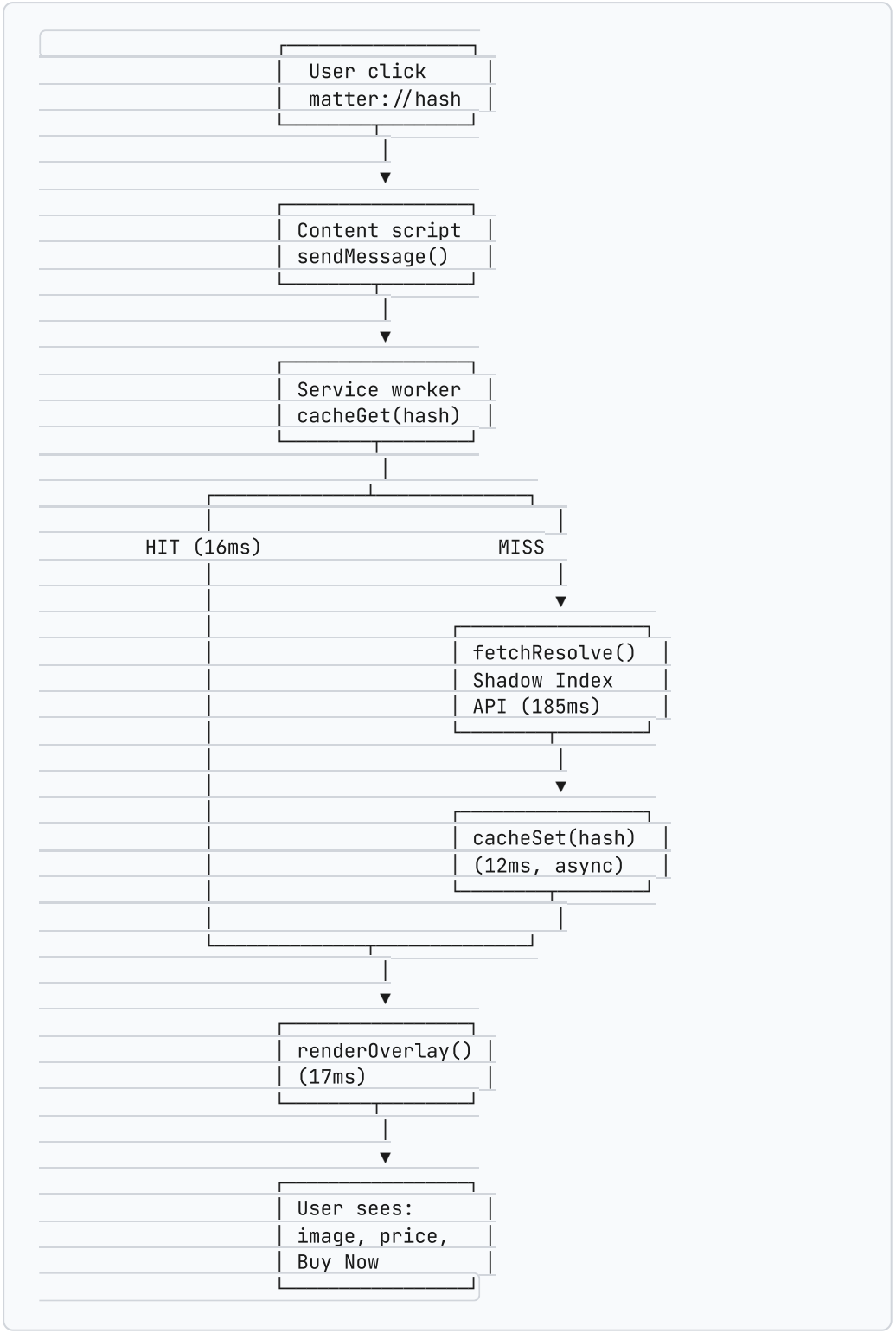
Metric	Result	Notes
Speedup factor	**11.5x**	Cache hit vs API path
Storage usage	**0.088%** of quota	1.8 MB for 100 entries
Memory	**0.2 MB** delta	20 clicks; zero leaks (heap snapshot verified)

10.5 Future Performance (CLIP Binary — Predicted)

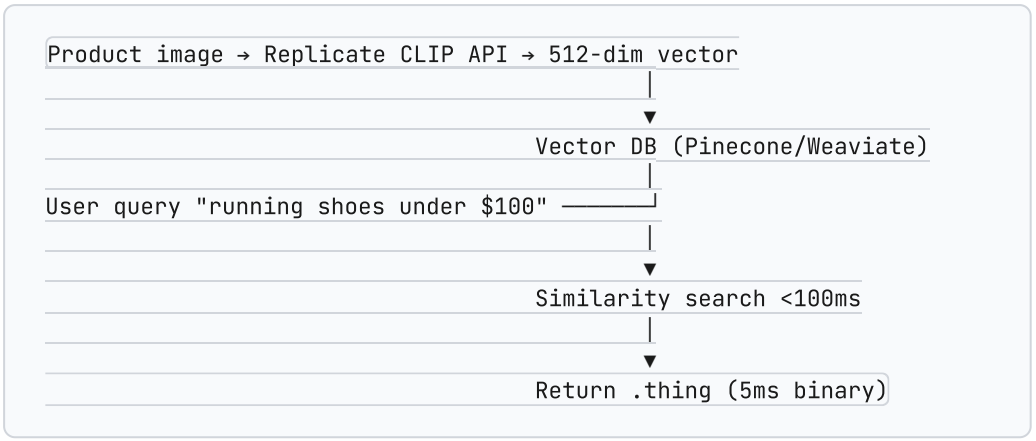
Metric	HTML/JSON	CLIP Binary	Improvement
Latency	700ms	5ms	140x faster
Payload size	50–200 KB	3–8 KB	20x smaller
Token consumption	2000/product	50/product	40x fewer
Cost (100 products)	$4.00	$0.10	97.5% savings

10.6 Architecture Diagrams

Data flow: Cache hit vs miss



Future: CLIP semantic search flow


Product image → Replicate CLIP API → 512-dim vector
Vector DB (Pinecone/Weaviate)
User query "running shoes under $100"
Similarity search <100ms
Return .thing (5ms binary)

10.7 References

Document	Purpose
docs/PROGRESS-AND-VERSION.md	Version history, release notes
docs/THURSDAY-INTEGRATION-VALIDATION.md	E2E validation, message protocol
docs/THURSDAY-PERFORMANCE-BASELINES.md	Cache hit/miss, overlay, speedup, memory
docs/15-DELIVERY-TIMELINE.md	Phase 1–5 delivery milestones
docs/02-TECHNICAL-ARCHITECTURE.md	System architecture
docs/04-DEVELOPMENT-ROADMAP.md	Roadmap phases
docs/05-RISK-ASSESSMENT.md	Risk register
docs/06-BUDGET-FINANCIAL-PROJECTIONS.md	Financial projections
README.md	Project overview, setup
CODEBASE-ANALYSIS.md	Codebase structure

Git: 8edb113, 1809686, 41585b0, ad05ed8, 5ebd2c6, a779c49, f4db192

PDF Conversion Instructions

To generate a print-ready, visually professional PDF from this Markdown file:

Option 1: md-to-pdf with Chrome (recommended — best formatting, diagrams, tables)

```
# Run from project root; uses docs/pdf-styles.css for profession
.\scripts\generate-technical-doc-pdf.ps1
```

Or manually:

```
$env:PUPPETEER_EXECUTABLE_PATH = "C:\Program Files\Google\Chrome
npx md-to-pdf THINGS-THAT-MATTER-TECHNICAL-DOCUMENTATION.md --st
```

Option 2: VS Code / Cursor
Use the "Markdown PDF" extension (by yzane) and export to PDF for similar quality.

Option 3: Pandoc (requires LaTeX: MiKTeX or TeX Live)

```
pandoc THINGS-THAT-MATTER-TECHNICAL-DOCUMENTATION.md -o THINGS-T
```

Option 4: Online
Upload to dillinger.io or markdown2pdf.com.

Document generated February 13, 2026. For CEO self-study, investor discussions, and technical recruiting.